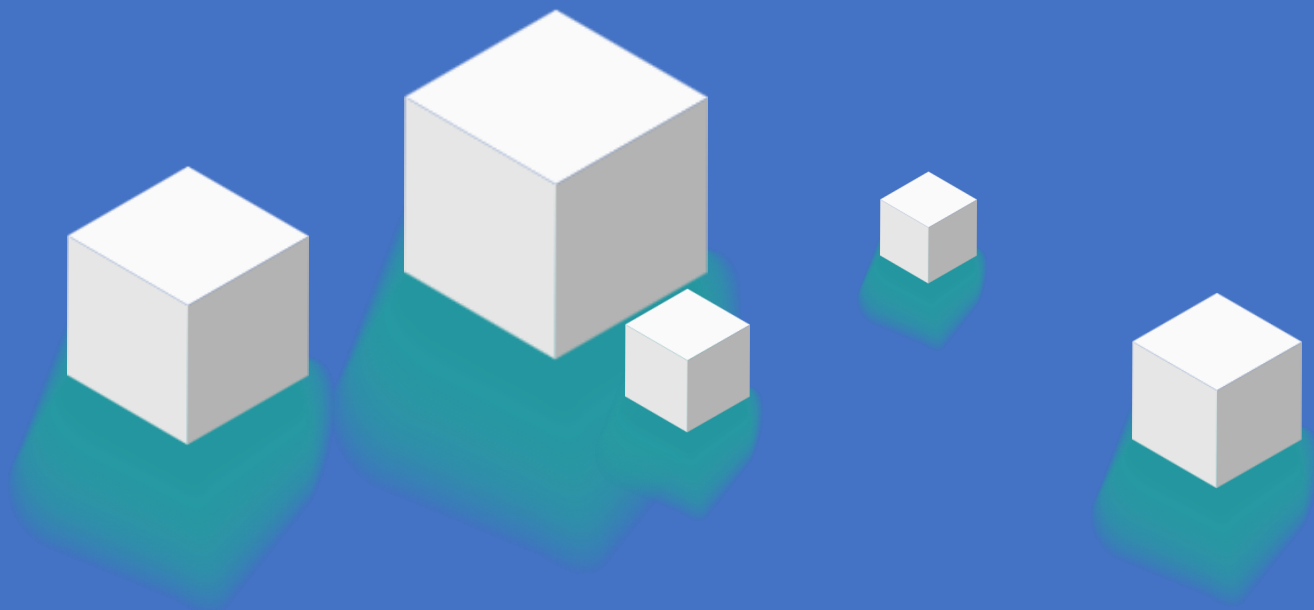


海龟交易法则



>> 今天的学习目标

海龟交易法则

- 经典海龟交易策略

核心原理

海龟的四大核心组件

CASE：经典海龟策略实战

仓位管理的方法

- 海龟交易策略进阶

CASE：ADX海龟策略

CASE：多周期海龟策略

CASE：ML增强海龟策略

经典海龟交易策略

核心原理

Thinking: 前面讲过的双均线、MACD策略, 都在解决 何时买入、何时卖出 的问题。那如果我已经有了一个不错的信号, 买入卖出的时机都确定了, 还有什么更重要的事情要做?

仓位管理和风险控制。海龟交易法则的创新, 不在于它的入场信号(唐奇安通道甚至不如MACD复杂), 而在于它把风险管理前置为策略设计的核心

维度	双均线/MACD	海龟交易法则
信号逻辑	指标交叉(滞后确认)	价格突破(直接捕捉)
核心哲学	趋势确认后跟随	趋势启动时捕捉
风险管理	固定仓位或简单止损	波动率自适应仓位(ATR)
出场机制	反向交叉或固定止盈	反向突破或跟踪止损
关注重点	什么时候买	买多少

从均线到通道: 突破的哲学

Thinking: 均线策略的底层逻辑是什么?

均线是一个低通滤波器, 过滤日常噪音, 保留趋势信号。

均线策略(如MACD)的入场逻辑:

- 计算两条均线的距离变化
- 等待金叉确认趋势已经形成
- 然后跟随趋势入场

Thinking: 均线策略 (比如MACD) 有什么问题?

均线策略永远是后知后觉的。当金叉发生时, 趋势可能已经走了一大截。

从均线到通道: 突破的哲学

Thinking: 海龟如何解决这个问题?

海龟交易法则换了一个思路: 不等趋势确认, 直接赌突破。

Thinking: 什么叫突破?

- 如果今天的收盘价, 比过去20天里任何一天的最高价都高 -> 这是一个向上突破, 买入
- 如果今天的收盘价, 比过去10天里任何一天的最低价都低 -> 趋势结束, 卖出

这就是唐奇安通道(Donchian Channel), 发明者Richard Donchian被称为趋势跟踪之父。

从均线到通道: 突破的哲学

Thinking: 突破入场比均线金叉快, 这是优势。但它的代价是什么?

代价是假突破。

价格突破了20日最高价, 可能只是短暂冲高, 随后就掉头回落。

=> 这就是为什么海龟需要严格的止损规则

即每一次入场都做好了可能马上被打脸的准备。

海龟的四大核心组件

海龟交易法则是一个完整的交易系统, 由四个组件紧密配合:

组件1: 唐奇安通道——何时行动

```
# 入场通道: 过去20日最高价
entry_high = bt.ind.Highest(self.data.high, period=20)

# 出场通道: 过去10日最低价
exit_low = bt.ind.Lowest(self.data.low, period=10)

# 入场: 突破20日最高价
if close > entry_high[-1]:
    self.buy()

# 出场: 跌破10日最低价
if close < exit_low[-1]:
    self.close()
```

Thinking: 为什么入场用20日, 出场只用10日?

不对称设计。

入场用较长的周期(20日), 是因为趋势启动需要更强的确认信号。

出场用较短的周期(10日), 是因为趋势逆转时, 我们想更快离场, 保护已有利润。

=> 入场要审慎, 出场要果断

海龟的四大核心组件

组件2: ATR(N值) —— 市场有多疯

ATR(Average True Range)是市场的心率仪。

真实波幅 = $\max(\text{今日最高}-\text{最低}, |\text{今日最高}-\text{昨收}|, |\text{今日最低}-\text{昨收}|)$

ATR = 20日真实波幅的移动平均

- 茅台的ATR可能是30元(假设股价1400元, 占比2.14% => 波动小, 白马股风格)
- 中芯国际的ATR可能是8元(假设股票110元, 占比7.27% => 波动大, 成长股风格)

=> ATR不判断方向, 只衡量波动幅度

海龟将ATR称为N值。在原始的海龟法则中, N值是整个系统的度量衡。即所有的仓位大小、加仓间距、止损距离, 都用N值来计算。

海龟的四大核心组件

组件3: ATR仓位管理 —— 买多少 (核心创新)

ATR是最重要的组件, 也是区别于其他趋势策略的根本。

```
def _calc_unit_size(self):  
    """  
    ATR仓位公式:  
    单位大小 = (账户总值 * 风险比例) / ATR  
    取整到100股(A股1手)  
    """  
    portfolio_value = self.broker.getvalue()  
    atr_val = self.atr[0]  
    if atr_val <= 0:  
        return 0  
    unit_size = (portfolio_value * self.p.risk_pct) / atr_val  
    unit_size = int(unit_size // 100) * 100  
    return max(unit_size, 100)
```

假设: 账户100万元, 风险比例 1% ($\text{risk_pct} = 0.01$)

茅台ATR: 30元, 中芯ATR: 8元

茅台 (1400元股价, $\text{ATR}=30$)

$\text{unit_size} = (1,000,000 \times 0.01) / 30$

$= 10,000 / 30$

$= 333.33$ 股

→ 取整到100股 = 300股

→ 名义市值 = $300 \times 1400 = 42$ 万元

中芯 (110元股价, $\text{ATR}=8$)

$\text{unit_size} = (1,000,000 \times 0.01) / 8$

$= 10,000 / 8$

$= 1,250$ 股

→ 取整到100股 = 1,200股

→ 名义市值 = $1,200 \times 110 = 13.2$ 万元

海龟的四大核心组件

Thinking: 茅台只买300股, 中芯买1200股, 这意味着什么?

不管你交易哪只股票, 价格波动1个ATR时, 你的账户恰好变动1%(即1万元)。

=> 这就是 风险恒定 的含义:

- 高波动股票(ATR大) -> 自动少买, 控制风险
- 低波动股票(ATR小) -> 自动多买, 不浪费资金

Thinking: 如果不用ATR仓位, 像之前MACD策略那样固定仓位95%, 会怎样?

在高波动股票上, 95%仓位意味着一个ATR的波动就可能让账户暴跌5%-10%。

在低波动股票上, 95%仓位可能又太保守, 浪费了资金利用率。

固定仓位的问题是: 对所有股票一视同仁, 但市场不会一视同仁地对待你。

海龟的四大核心组件

组件4: 金字塔加仓 —— 趋势确认后加码

海龟不会一次性全仓, 而是分批建仓:

规则:

- 最多持有4个单位
- 每上涨0.5个ATR, 加一个单位
- 每次加仓后, 止损线上移至 最新入场价 - 2*ATR

金字塔加仓逻辑

```
if self.units < self.p.max_units:
    if close >= self.last_add_price + self.p.add_n * atr_val:
        size = self._calc_unit_size()
        cash = self.broker.getcash()
        if size > 0 and cash > close * size * 1.01:
            self.order = self.buy(size=size)
```

海龟的四大核心组件

金字塔加仓的博弈本质:

- 第1单位入场 -> 试探性进入, 用最小的风险验证趋势
- 上涨0.5N -> 趋势初步确认, 加第2单位
- 继续上涨0.5N -> 趋势进一步确认, 加第3单位
- 再涨0.5N -> 趋势强势, 加第4单位(满仓)

金字塔加仓逻辑

```
if self.units < self.p.max_units:
    if close >= self.last_add_price + self.p.add_n * atr_val:
        size = self._calc_unit_size()
        cash = self.broker.getcash()
        if size > 0 and cash > close * size * 1.01:
            self.order = self.buy(size=size)
```

如果突破后立即下跌 -> 只有第1单位的损失(总资金的1%)

如果突破后大幅上涨 -> 4个单位全部盈利, 利润放大

=> 这就是 用小亏验真假, 用大仓吃趋势 的策略思维。

CASE： 经典海龟策略实战

TO DO： 搭建经典海龟策略

为了直观展示ATR仓位管理的价值, 我们做一个对照实验:

简单海龟: 只有唐奇安通道突破信号, 仓位固定95%(和之前MACD策略一样)

完整海龟: 唐奇安通道 + ATR仓位管理 + 金字塔加仓 + 2N止损

两者用完全相同的买卖信号, 唯一的区别是仓位管理方式。

CASE：经典海龟策略实战

```
class SimpleTurtleStrategy(bt.Strategy):
    """简单版海龟: 只有信号, 没有仓位管理"""
    params = (('entry_period', 20), ('exit_period', 10))

    def __init__(self):
        self.entry_high = bt.ind.Highest(self.data.high, period=self.p.entry_period)
        self.exit_low = bt.ind.Lowest(self.data.low, period=self.p.exit_period)

    def next(self):
        if not self.position:
            if self.data.close[0] > self.entry_high[-1]:
                self.buy() # 全仓买入(95%)
        else:
            if self.data.close[0] < self.exit_low[-1]:
                self.close()
```

指标	简单海龟(固定95%仓位)	完整海龟(ATR仓位)
总收益	20.89%	9.37%
最大回撤	17.23%	13.67%
夏普比率	2.08	0.59
交易次数	9	9
胜率	33.30%	33.30%
盈亏比	5	3.62

CASE：经典海龟策略实战

Thinking: 简单海龟收益更高(+20.89% > +9.37%), 那ATR仓位管理有什么用?

看似矛盾, 其实恰好说明了一个深刻的道理:

- 简单海龟仓位大(95%), 趋势来了赚得多, 但假突破时也亏得猛。最大回撤17.23%
- 完整海龟仓位小(ATR控制), 每笔交易只冒1%的风险。单次收益少, 但扛打能力强。

这个区间(2024-2025)沪深300ETF总体上涨, 简单海龟大仓位踩中了大趋势, 所以收益高。

但如果换一个下跌区间, 大仓位就是灾难。

Thinking: 你更喜欢简单海龟, 还是经典海龟?

海龟的仓位管理目标不是最大化收益 => 而是让我们活得够久, 能看到下一次大趋势。

CASE：经典海龟策略实战

不同市场环境对比：

趋势跟踪策略有一个隐含前提：市场要有趋势。

我们选择三只不同风格的标的，看完整海龟策略的表现（时间范围：2024-01-01到2025-12-31）

标的	策略	总收益	最大回撤	夏普	胜率	盈亏比
贵州茅台	买入持有	-12.76%	--	--	--	--
	简单海龟	-18.10%	22.88%	-1.37	14.30%	0.92
	完整海龟	-19.10%	21.19%	-5.22	11.10%	0.1
沪深300ETF	买入持有	46.06%	--	--	--	--
	简单海龟	20.89%	17.23%	2.08	33.30%	5
	完整海龟	9.37%	13.67%	0.59	33.30%	3.62
纳指ETF	买入持有	54.95%	--	--	--	--
	简单海龟	43.23%	11.39%	7.74	100%	--
	完整海龟	19.39%	7.00%	3.6	75.00%	2.17

CASE: 经典海龟策略实战

Thinking: 为什么茅台的海龟策略是亏的, 甚至比买入持有还差?

2024-2025年茅台处于下跌趋势。海龟策略在下跌中会频繁产生假突破:

- 价格短暂突破20日最高价(海龟买入)
- 随后迅速回落并跌破止损线(海龟止损)
- 反复进出, 每次亏1%-2%, 累计亏损严重

=> 这就是震荡市的绞肉机, 即趋势策略在没有趋势的市场中会被反复割韭菜。

纳指ETF表现最好: 夏普3.60, 胜率75%, 回撤仅7%。因为纳指2024-2025年有明确的上升趋势

茅台最差: 下跌趋势中不断假突破止损, 亏损累累

趋势策略的真相: 选择合适的市场, 比优化参数重要100倍

仓位管理的数学直觉

Thinking: 风险管理和买卖信号哪个更重要?

风险管理!

假设你有100万, 亏损50% -> 剩50万。

此时你需要赚多少才能回到100万?

=> 不是50%, 而是100%!

爆仓的非对称性, 即亏损和盈利是不对称的

亏损	回本需要的涨幅
10%	11.10%
20%	25.00%
30%	42.90%
50%	100.00%
70%	233.30%

仓位管理的数学直觉

这就是为什么海龟法则把止损设在2N(大约2-4%的账户亏损):

- 每笔交易最多亏2%(单位风险1%, 极端情况下2单位同时止损)
- 即使连续止损5次, 总亏损也只有10%左右
- 回本只需要11.1%的涨幅, 完全可以在一次成功的趋势中赚回来

反过来, 如果每次满仓操作、每次亏10-20%:

- 3次连续止损就可能亏掉50%
- 需要翻倍才能回本, 心态也早就崩了

亏损	回本需要的涨幅
10%	11. 10%
20%	25. 00%
30%	42. 90%
50%	100. 00%
70%	233. 30%

仓位公式的优雅

海龟仓位公式: $\text{单位大小} = (\text{账户资金} * 1\%) / \text{ATR}$

这个公式本质上是在说:

- 每笔交易, 我最多愿意承受账户资金1%的波动
- 波动大的股票, 少买一点
- 波动小的股票, 多买一点
- 无论哪只股票, 风险水平都是统一的

=> 风险预算的思想, 也是现代量化基金 风险平价(Risk Parity) 策略的鼻祖。

Summary

海龟四大组件:

- 唐奇安通道: 突破20日高入场, 跌破10日低出场
- ATR(N值): 20日真实波幅均值, 衡量市场波动
- ATR仓位: $\text{单位} = (\text{资金} * 1\%) / \text{ATR}$ (风险恒定)
- 金字塔加仓: 最多4个单位, 每涨0.5N加仓, 2N止损

```
# 唐奇安通道
```

```
entry_high = bt.ind.Highest(self.data.high, period=20)
```

```
exit_low = bt.ind.Lowest(self.data.low, period=10)
```

```
# ATR仓位
```

```
unit_size = (portfolio_value * 0.01) / atr
```

```
unit_size = int(unit_size // 100) * 100 # A股取整到100股
```

```
# 金字塔加仓条件
```

```
if close >= last_add_price + 0.5 * atr:
```

```
    self.buy(size=unit_size) # 加一个单位
```

```
# 止损
```

```
stop_price = entry_price - 2 * atr
```

Summary

核心公式:

单位大小 = (账户资金 * 风险比例) / ATR

=> 价格波动1个ATR, 账户恰好变动 risk_pct

维度	传统思维	海龟思维
核心问题	什么时候买?	买多少?
仓位逻辑	固定比例 (如95%)	波动率自适应 (ATR)
风险控制	亏X%止损 (固定)	亏2N止损 (自适应)
加仓方式	一次性全仓	金字塔分批建仓
目标	最大化收益	控制风险, 活得更久

- 简单海龟(大仓位)和完整海龟(ATR仓位)在同一信号下, 回撤差异明显
- 趋势跟踪策略在有趋势的市场中表现优异, 在震荡/下跌市场中频繁亏损
- ATR仓位管理不追求最大化收益, 而是让账户在连续止损后仍然有足够弹药
- 选择合适的交易标的(存在趋势), 比优化参数更重要

CASE: ADX海龟策略

经典海龟的痛点

Thinking: 经典海龟策略的痛点是什么吗?

在纳指ETF(明确上涨趋势)上表现优异: 夏普3.60

在茅台(下跌趋势)上惨败: 夏普-5.22

=> 核心矛盾: 假突破太多

Thinking: 经典海龟在茅台上的9笔交易中, 只有1笔盈利(胜率11.1%)。剩下8次都是假突破止损。

有没有办法在入场前先判断 这次突破是真的还是假的?

- **ADX过滤:** 趋势不够强时拒绝入场
- **多周期过滤:** 大趋势方向不对时拒绝入场
- **机器学习:** 用历史数据训练模型, 预测突破成功率

CASE: ADX海龟策略

Thinking: 在自适应策略中, ADX是如何使用的?

ADX(Average Directional Index): 衡量趋势的强度, 不分方向

- $ADX > 25$: 强趋势(不管是暴涨还是暴跌)
- ADX 15-25: 弱趋势
- $ADX < 15$: 基本无趋势(横盘震荡)

=> ADX可以用来切换策略(趋势市用MACD, 震荡市用RSI)。

在海龟中, ADX可以用来过滤入场信号 => 用ADX识别市场状态。

CASE: ADX海龟策略

Thinking: ADX如何计算?

ADX计算基于DM (Directional Movement) 指标, 分为三步:

Step1. 计算+DM和-DM

+DM = 今日高点 - 昨日高点 (如果 结果>0且大于|-DM|, 则取该值, 否则为0)

-DM = 昨日低点 - 今日低点 (如果 结果>0且大于+DM, 则取该值, 否则为0)

每天只保留一个方向上的突破力度 => 谁大听谁的。

Step2. 计算DX (Directional Index, 方向指数)

+DI = 平滑后的+DM / ATR × 100 (代表多头力量占比)

-DI = 平滑后的-DM / ATR × 100 (代表空头力量占比)

DX = $|+DI - (-DI)| / (+DI + -DI) \times 100$ (代表多空力量的差距程度)

DX衡量多空双方谁占绝对优势 (0=均衡, 100=一边倒)

CASE：ADX海龟策略

第三步：计算ADX (Average DX)

ADX = DX的N周期平滑移动平均(通常N=14)

给DX打平均分，过滤单日噪音，看趋势强度的持续性。

假设：

14日平滑+DM = 50（多头动能累计）

14日平滑-DM = 30（空头动能累计）

ATR = 2（平均真实波幅）

+DI计算： $50 \div 2 \times 100 = 25$ ，即多头强度25%

-DI计算： $30 \div 2 \times 100 = 15$ ，即空头强度15%

DX = $(25-15) \div (25+15) \times 100 = 25$ ，即多空差距25%

ADX 前13日DX均值≈25，今日DX=25 → 25，代表 趋势强度中等

ADX值	市场状态	策略选择
> 25	强趋势（单边市）	用趋势策略（MACD、均线、海龟）
15-25	弱趋势（过渡市）	减仓观望，或收紧止损
< 15	无趋势（震荡市）	用震荡策略（RSI、布林带、网格

ADX只看 有没有趋势，不看向哪走；
如果要看方向，看+DI和-DI谁大。

CASE: ADX海龟策略

在海龟的入场逻辑前, 加一个ADX检查:

```
class ADXTurtleStrategy(bt.Strategy):
    params = (
        # ... 海龟原始参数不变 ...
        ('adx_period', 14), ('adx_threshold', 15),
    )

    def _calc_adx(self):
        """用talib计算当前ADX值"""
        size = len(self.data)
        h = np.array(self.data.high.get(size=size), dtype=np.float64)
        l = np.array(self.data.low.get(size=size), dtype=np.float64)
        c = np.array(self.data.close.get(size=size), dtype=np.float64)
        adx = talib.ADX(h, l, c, timeperiod=self.p.adx_period)
        return adx[-1] if not np.isnan(adx[-1]) else 0.0
```

```
def next(self):
    # ... ATR检查 ...
    if not self.position:
        # ADX门卫: 趋势不够强时拒绝入场
        adx_val = self._calc_adx()
        if adx_val < self.p.adx_threshold:
            return # 拒绝入场
        if c > self.entry_high[-1]:
            s = self._calc_unit_size()
            if s > 0: self.order = self.buy(size=s)
    else:
        # 已持仓: 出场/加仓逻辑不变, ADX只影响开仓决策
        # ... 止损、出场、加仓 ...
```

CASE: ADX海龟策略

Thinking: ADX阈值设多少合适?

经过测试, 阈值15是更好的选择:

- $ADX > 25$ 太严格: 会过滤掉趋势刚启动时的信号(此时ADX还没升到25, 但趋势已经在酝酿)
- $ADX > 15$ 更宽松: 只过滤最差的 完全没有趋势 的假突破, 不伤害好信号

=> 简单规则往往最有效, 不要过度优化

CASE：ADX海龟策略

回测结果: ADX过滤的效果

标的	买入持有	经典海龟	ADX海龟	收益变化	回撤变化	交易变化
平安银行 (601318)	98.40%	8.29%	25.73%	17.44%	-1.03%	10->7
沪深300ETF (510300)	46.10%	9.37%	9.23%	-0.14%	0%	9->8
贵州茅台 (600519)	-12.80%	-19.10%	-18.97%	0.12%	-2.21%	9->9

平安银行的详细对比

指标	经典海龟	ADX海龟
总收益	8.29%	25.73%
最大回撤	14.53%	13.50%
夏普比率	0.82	2.26
交易次数	10	7
胜率	40.00%	57.10%
盈亏比	1.6	3.27
利润因子	1.28	6.54

CASE: ADX海龟策略

Thinking: ADX过滤在平安银行上效果惊人, 交易从10次=> 7次, 但收益+8.29% -> +25.73%, 说明什么?

说明那被过滤掉的3次交易, 全是低质量的假突破。

ADX < 15时市场没有趋势方向, 此时的突破大概率是噪音。去掉这些噪音交易后, 留下的都是真正有趋势支撑的突破 -> 胜率从40%提升到57.1%, 盈亏比从1.60提升到3.27。

沪深300ETF: ADX过滤几乎没有影响(减少1笔交易, 收益基本不变)。这说明300ETF的突破质量本身就不错, ADX过滤 没有伤害好信号。

茅台: 效果微弱(都在亏)。这是因为茅台处于下跌趋势, ADX无法挽救方向性错误 -- 趋势向下时, ADX值反而可能很高(下跌趋势也是趋势)。

ADX > 15 是一个极低成本过滤器: 不伤害好信号, 但能过滤最差的假突破

最佳效果出现在 有趋势但趋势不算很强 的股票上(如平安银行)

ADX无法解决方向性问题: 在持续下跌的市场中, 需要其他手段(如多周期过滤)

CASE：多周期海龟策略

CASE：多周期海龟策略

TO DO：搭建多周期融合海龟

大周期过滤, 小周期执行

ADX告诉我们市场有没有趋势, 但不告诉我们趋势的方向。

多周期策略解决了方向问题:

- 周线(大周期): 判断大趋势的方向 -> 只在大趋势向上时做多
- 日线(小周期): 寻找精确的入场点 -> 用唐奇安通道找突破

CASE：多周期海龟策略

Backtrader 提供了 resampleddata API, 可以将日线数据重采样为周线:

```
def run_multi_tf_backtest(strategy_class, stock_code, start_date, end_date, kwargs):  
    df = load_stock_data(stock_code, start_date, end_date)  
    cerebro = bt.Cerebro()  
  
    # data0: 日线数据  
    data_daily = bt.feeds.PandasData(dataname=df)  
    cerebro.adddata(data_daily)  
  
    # data1: 周线数据(从同一份日线数据重采样)  
    data_weekly = bt.feeds.PandasData(dataname=df)  
    cerebro.resampleddata(data_weekly, timeframe=bt.TimeFrame.Weeks)  
  
    # 策略中 self.data0 = 日线, self.data1 = 周线  
    cerebro.addstrategy(strategy_class, kwargs)
```

CASE：多周期海龟策略

周线趋势判断：

```
def _get_weekly_trend(self):
    """判断周线大趋势: up / down / neutral"""
    wc = self.data1.close[0]    # 周线收盘价
    wh = self.weekly_high[-1]   # 周线8周最高价
    wl = self.weekly_low[-1]    # 周线8周最低价
    if wc > wh: return 'up'     # 突破周线高点 -> 大趋势向上
    if wc < wl: return 'down'   # 跌破周线低点 -> 大趋势向下
    return 'neutral'           # 中间区域 -> 方向不明确
```

条件	结果	含义
$wc > wh$	up	本周收盘突破了前8周最高价，大趋势向上
$wc < wl$	down	本周收盘跌破了前8周最低价，大趋势向下
其他	neutral	价格在8周通道内部，方向不明

CASE：多周期海龟策略

过滤规则(宽松设计):

```
def next(self):
    weekly_trend = self._get_weekly_trend()
    if not self.position:
        # 只有周线趋势明确向下时, 才禁止做多
        if weekly_trend == 'down':
            return # 拒绝入场
        # up 或 neutral 都允许入场
        if c > self.daily_entry_high[-1]:
            self.buy(size=self._calc_unit_size())
    else:
        # 持仓时: 周线转为下跌 -> 强制平仓
        if weekly_trend == 'down':
            self.close()
            return
        # ... 日线止损/出场/加仓 ...
```

CASE：多周期海龟策略

Thinking: 为什么周线过滤只挡down, 不要求必须是up才入场?

如果要求周线必须是up才允许日线入场, 会过于严格。

因为周线突破8周高点是一个很高的门槛, 大部分时间周线都处于neutral状态。

如果只在up时交易, 会错过大量机会, 尤其是趋势刚刚启动、周线还没突破时的第一波行情。

=> 设计为: 只挡最差的情况(周线明确下跌), 其他情况都允许。

这是一个保守的安全网, 不是积极的信号选择器。

CASE：多周期海龟策略

回测结果：

510300 沪深300ETF, 2024-2025

指标	单周期(仅日线)	多周期(周线+日线)
总收益	9.37%	6.83%
最大回撤	13.67%	13.67%
夏普比率	0.59	0.43
交易次数	9	9
胜率	33.30%	22.20%
盈亏比	3.62	5.52

Summary

Thinking: 多周期策略的收益反而低了, 那它有价值么?

- 在这个特定的回测区间, 多周期过滤确实降低了收益。
- 盈亏比提升: 从3.62到5.52, 意味着每笔盈利交易的利润更大, 更划算
- 多周期不一定更好, 但是大周期过滤, 小周期执行的思维框架, 很多专业交易者使用这种方法
- 多周期策略在更长的回测周期(如3-5年)和指数类标的上通常效果更好

CASE: ML增强海龟策略

CASE: ML增强海龟策略

TO DO: 搭建ML增强海龟策略

为什么引入机器学习? ADX过滤是一个硬规则, 即 只看一个指标(ADX)决定是否入场。

但影响突破成功的因素很多:

- 波动率在扩大还是收缩?
- 趋势强度如何?
- 成交量是否放大?
- 股价之前盘整了多久?
- RSI处于什么位置?

Thinking: 如果能综合考虑这些因素, 用历史数据学习 什么样的突破容易成功, 是不是能做出更好的判断?

=> 这是机器学习的价值, 即从历史的突破事件中, 学习哪些特征组合预示着 真突破。

CASE: ML增强海龟策略

流程步骤:

Step 1: 特征工程 (提取突破事件的市场特征)

Step 2: 模型训练 (LightGBM, 严格时间分割)

Step 3: 生成预测 (为目标股票的每个突破点计算成功概率)

Step 4: 回测对比 (ML海龟 vs 经典海龟)

CASE: ML增强海龟策略

Step 1: 特征工程

在每一个突破事件(收盘价 > 20日最高价)发生时, 提取8个市场特征:

```
features = {  
    'atr_ratio':    atr / close,      # 归一化波动率  
    'adx':          adx,             # 趋势强度(0-100)  
    'vol_ratio':    volume / vol_ma_20, # 量比(放量突破更可靠)  
    'rsi':          rsi,             # 相对强弱(避免追高)  
    'breakout_strength': (close - channel_high) / atr, # 突破力度  
    'momentum_5d':   close / close_5d_ago - 1, # 5日动量  
    'consolidation_days': days_since_last_breakout, # 盘整天数  
    'atr_change':    atr / atr_5d_ago - 1, # 波动率变化率  
}
```

CASE：ML增强海龟策略

标签定义(什么算 真突破):

- 突破后5个交易日内, 价格最高涨幅 > 2% -> 标签=1(真突破)
- 否则 -> 标签=0(假突破)

Thinking: 选取的这些特征是否合理?

=> 每个特征都有其交易逻辑

注意：所有特征都进行了归一化处理(比如用ATR/Close，而不是ATR绝对值)
=> 归一化的目的，是用于不同股票之间横向比较。

特征	逻辑
atr_ratio	归一化波动率，跨股票可比
adx	趋势越强，突破越可能延续
vol_ratio	放量突破比缩量突破更可靠
rsi	RSI过高时追涨风险大
breakout_strength	突破力度越大越可靠
momentum_5d	有动量支撑的突破更强
consolidation_days	盘整越久，突破后的能量越大
atr_change	波动率正在扩大 = 趋势正在启动

CASE: ML增强海龟策略

Step 2: 多股票训练 + 时间分割

Thinking: 如果只用一只股票训练, 2年内可能只有20-30个突破事件, 这么少的样本能训练出可靠的模型吗?

不能。所以我们用多股票训练: 从6只不同行业的股票中收集突破事件, 扩大样本量。

```
train_stocks = [  
    ('600519.SH', '贵州茅台'), # 消费白马  
    ('300750.SZ', '宁德时代'), # 新能源成长  
    ('510300.SH', '沪深300ETF'), # 宽基指数  
    ('688981.SH', '中芯国际'), # 科技半导体  
    ('601318.SH', '平安银行'), # 金融  
    ('159941.SZ', '纳指ETF'), # 海外市场  
]  
# 合计: 174个突破事件 (比单只股票的25个多了6倍)
```

CASE: ML增强海龟策略

时间分割(避免未来数据泄露):

2024年数据 -> 训练集

2025年数据 -> 测试集

针对时间序列数据 => 训练集要在测试集之前。

不能用随机拆分(sklearn的train_test_split) => 未来数据会泄露到训练集中。

CASE: ML增强海龟策略

模型选择(防过拟合):

```
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=80,    # 树的数量(不要太多)
    max_depth=3,       # 浅树(每棵树最多3层, 防止过拟合)
    learning_rate=0.1,  # 学习率
    min_child_samples=3, # 最小叶节点样本数
    reg_alpha=0.1,      # L1正则化
    reg_lambda=1.0,     # L2正则化
    is_unbalance=True,  # 处理类别不平衡
    verbose=-1,
)
```

Thinking: 为什么用LightGBM而不是深度学习?

- **样本量小(174个事件):** 深度学习需要大量数据, 这里样本太少, 深度学习会严重过拟合
- **结构化表格数据:** GBDT类模型(LightGBM/XGBoost)在结构化数据上的表现通常优于深度学习
- **可解释性:** LightGBM可以输出特征重要性 => 哪些因素影响突破成功率

CASE: ML增强海龟策略

模型评估:

准确率 51.6%

精确率 44.4%

召回率 38.1%

F1分数 41.0%

Thinking: 准确率只有51.6%, 和抛硬币差不多, 这个模型有用吗?

有用!

- **不要被准确率迷惑:** 在交易中, 我们不需要100%准确, 只需要精确率 > 随机水平即可。精确率44.4%意味着模型说 买 的时候, 有44.4%的概率是对的
- **关键是过滤垃圾信号:** 模型不需要找出所有真突破, 只需要过滤掉最差的假突破。即使召回率只有38.1%(错过了一些真突破), 只要留下的信号质量更高, 整体收益就会改善
- **和ADX过滤的思路一样:** 宁可错过一些机会, 也不要再在垃圾信号上亏钱

CASE： ML增强海龟策略

特征重要性:		Thinking: 特征重要性揭示了什么?
atr_change	1.00	• atr_change(波动率变化):
adx	0.80	波动率正在扩大 = 市场从沉寂中苏醒, 趋势可能正在启动
consolidation_days	0.51	• adx(趋势强度):
atr_ratio	0.44	趋势越强, 突破越可能延续(和CASE 2的ADX过滤呼应)
rsi	0.42	• consolidation_days(盘整天数):
breakout_strength	0.36	盘整越久, 积蓄的能量越大, 突破后越可能走出大行情
momentum_5d	0.26	=> 这三个特征组合起来的含义
vol_ratio	0.21	在充分盘整后, 波动率开始扩大, 且趋势已经形成的突破, 最可能成功。

CASE：ML增强海龟策略

回测对比：ML增强海龟 vs 经典海龟, 平安银行(601318), 2024-2025

```
class MLTurtleStrategy(bt.Strategy):
    def next(self):
        if not self.position:
            if c > self.entry_high[-1]:
                # 查询ML预测概率
                prob = self.p.predictions.get(current_date, 0.0)
                if prob >= self.p.ml_threshold: # 概率 >= 0.5 才入场
                    s = self._calc_unit_size()
                    if s > 0: self.order = self.buy(size=s)
                    self.ml_passed += 1
            else:
                self.ml_filtered += 1 # 被ML过滤
        else:
            # 出场/加仓逻辑完全不变
            # ...
```

指标	经典海龟	ML海龟	变化
总收益	8.29%	14.31%	6.02%
最大回撤	14.53%	9.05%	-5.48%
夏普比率	0.82	2.01	1.19
交易次数	10	7	-3
胜率	40.00%	42.90%	2.90%
盈亏比	1.6	3.18	1.58
利润因子	1.28	3.25	1.97

Summary

Thinking: ML过滤统计: 突破信号共18个, ML通过7个(39%), 过滤11个(61%)。

ML过滤了61%的信号, 只保留了39%, 但所有指标全面改善。这意味着什么?

被过滤掉的那61%的突破信号, 大部分是质量低的假突破。

ML模型通过综合8个特征, 识别出了 这次突破的市场环境不好 的情况, 并拒绝入场。

=> 留下的39% 是在 波动率扩大、趋势已形成、充分盘整 条件下的高质量突破。

=> 这就是传统量化 + AI 的典型结合方式:

- 传统策略(海龟)提供交易框架和风险管理
- AI(LightGBM)提供信号质量过滤
- AI不替代策略逻辑, 只增强信号质量

防过拟合的核心原则

Thinking: 什么是过拟合?

过拟合 = 模型记住了历史数据的噪音, 而不是学到了真正的规律。

通俗来说: 一个在历史回测中表现完美的策略, 实盘时可能一塌糊涂。

措施	做法	原因
时间分割	2024训练, 2025测试	避免未来数据泄露
多股票训练	6只跨行业股票	避免模型只记住单只股票的特征
浅树	max_depth=3	限制模型复杂度
正则化	reg_alpha=0.1, reg_lambda=1.0	惩罚过于复杂的模型
特征归一化	ATR/Close而非ATR	跨股票可比, 不依赖价格量级
最小叶节点	min_child_samples=3	防止在极少样本上做判断

防过拟合的核心原则

Thinking: 如何判断模型是否过拟合?

看训练集和测试集的差距:

- 如果训练集准确率99%, 测试集准确率50% -> 严重过拟合
- 如果训练集准确率60%, 测试集准确率52% -> 基本合理

=> 这里测试集准确率51.6%, 虽然不高, 但也说明模型没有在记忆历史

防过拟合的核心原则

Thinking：未来还有哪些进阶优化？

- Walk-Forward验证: 滚动窗口训练/测试, 比固定分割更稳健

步骤	操作	说明
1. 初始训练	用前N天数据训练模型	2015-01-01 至 2015-12-31
2. 预测1天	预测下1天（或1周）	预测 2016-01-04
3. 滚动窗口	加入真实数据，剔除最旧数据	训练窗口变为 2015-01-04 至 2016-01-04
4. 重复	循环直至最新日期	每天滚动一次

- 更多特征: 大盘状态(上证指数趋势)、行业轮动、资金流向
- 模型监控: 定期检查特征分布是否漂移, 及时重训模型
- 集成学习: 结合ADX过滤 + ML预测, 多层过滤

Walk-Forward = 模拟实盘运行：假设过去每一天我都按规则重新训练，结果会不会崩？它不是训练多个模型备用，而是验证"训练机制"本身是否靠谱。

Summary

三种优化方案对比

方案	原理	复杂度	效果	适用场景
CASE：ADX	ADX<15时拒绝入场	低	过滤最差假突破	通用，推荐首选
CASE：多周期	周线判方向，日线找入场	中	避免逆势交易	中长线，指数类
CASE：ML增强	多特征综合判断突破质量	高	全面提升信号质量	有数据和算力时

Summary

CASE 1 - 经典海龟(纳指ETF):

- 收益+19.4%, 回撤7.0%, 夏普3.60, 胜率75%
- 趋势明确时的海龟非常强

CASE 2 – ADX(平安银行):

- 经典 -> ADX: +8.29% -> +25.73%, 夏普0.82 -> 2.26
- ADX>15是极低成本的过滤器

CASE 3 - 多周期(沪深300ETF):

- 盈亏比从3.62提升到5.52
- 核心价值: Backtrader多周期API + 大局观思维

CASE 4 - ML增强(平安银行):

- 经典 -> ML: +8.29% -> +14.31%, 回撤14.53% -> 9.05%
- ML过滤61%低质量信号, 所有指标全面改善
- 最重要特征: 波动率变化 > 趋势强度 > 盘整天数

Summary

海龟原始概念	现代量化应用
ATR仓位管理	风险平价 (Risk Parity) 策略
金字塔加仓	CTA趋势跟踪基金
多品种分散	全球宏观策略
N值止损	目标波动率 (Target Volatility)
唐奇安通道	各种突破类策略的基础

思考

Thinking: 为什么经典海龟策略，有时候不如买入持有？

- ATR仓位管理在强趋势中反噬（最关键原因）

ATR仓位公式：单位大小 = (账户资金 * 1%) / ATR

这个公式有一个反直觉的特性：波动率越大，仓位越小。

而 2024年9-10月 沪深300暴涨那段行情，恰恰是波动率最大的时候。结果就是：

市场大涨时，ATR飙升 -> 策略反而减小仓位

趋势最强的那段行情，策略的仓位敞口最小

2024-10 那波从3.3涨到4.2的大行情，简单海龟（95%仓位）净值从1.0跳到1.3，而完整海龟只从1.0涨到1.2。

同样的信号，仓位不同，收益差了一倍。

思考

- 频繁假突破的锯齿损耗

9笔交易中只有3笔盈利（胜率33.3%）。在2024年上半年和2025年上半年的震荡期，唐奇安通道频繁触发假突破：

突破20日新高 -> 买入

很快跌破10日新低 -> 止损卖出

过几天又突破 -> 再买入...

每次进出都会产生实际亏损和手续费，而买入持有完全不受影响。

思考

- 空仓期错过行情

从净值曲线可以看到：策略在大量时间里是平线（空仓状态）。

2025年下半年沪深300从3.8涨到4.7这段 +24% 的行情，完整海龟的净值几乎没有变化

=> 说明策略在这个阶段基本处于空仓或极小仓位。

=> 买入持有的优势就在于永远满仓，不会错过任何上涨。

思考

- 金字塔加仓来不及建满

加仓规则是 每上涨0.5个ATR加一个单位，最多4个单位。但在快速拉升的行情中：

第1个单位刚入场

还没来得及加到第2、3、4个单位

价格已经回调，触发止损或出场信号

最终全仓位从未建立

思考

- 根本原因：策略的设计目标不同

海龟策略的核心理念不是赚最多，而是"活最久"。从最大回撤数据可以看出：

完整海龟回撤 13.67%，比简单海龟的 17.23% 更小

纳指ETF上，完整海龟回撤仅 7.00%

海龟策略是为多市场、长周期、分散化设计的

原版海龟交易员同时交易几十个品种（商品期货、外汇等），靠个别品种的大趋势盈利覆盖其他品种的小额亏损。单一标的、单一牛市的场景下，它天然不如买入持有。

打卡：海龟交易策略



针对你的自选股，使用海龟交易策略

- 使用自适应方法
- 使用ML增强方法
- 尝试修改ADX阈值(从15改为20或25), 观察对交易次数和收益的影响

Thinking：如果同时使用ADX过滤 + ML预测, 双重过滤会更好吗?

还是会过滤掉太多机会?



Thank You
Using data to solve problems